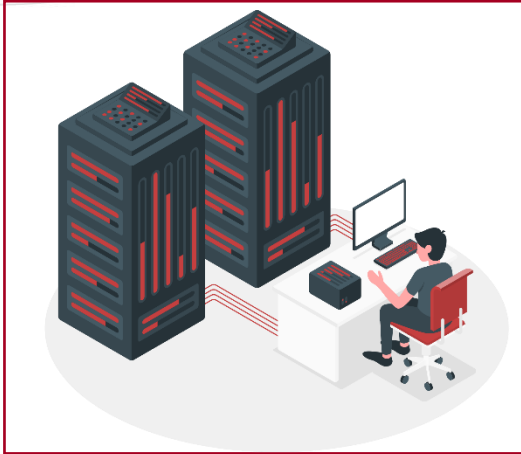




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente
Ingeniería en Desarrollo de Software

Asignatura: **Administración de Bases de Datos**

Unidad #4

Fecha: 29 de septiembre al 05 de octubre 2025

Tema: Operación y mantenibilidad.

Introducción:

Este material tiene como propósito introducirte en el manejo de bitácoras, transacciones y modos de operación dentro de un sistema de gestión de bases de datos (DBMS). Aprenderás cómo se registran y controlan las modificaciones a los datos, cómo se garantiza la recuperación ante fallos mediante el uso de comandos como COMMIT y ROLLBACK, y cuál es la función de las transacciones en la integridad del sistema. Además, se explicarán los diferentes modos de operación del DBMS (alta, baja y recuperación) y los comandos necesarios para iniciar o detener una base de datos de forma segura y controlada.

Sección 1: Bitácoras del trabajo del DBMS

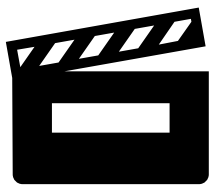
En un sistema de gestión de bases de datos (DBMS), las bitácoras de trabajo desempeñan un papel esencial, especialmente en entornos multiusuario, donde las bases de datos (BD) son por lo general más grandes, complejas y requieren un control más riguroso de las operaciones realizadas sobre los datos.

Entre las principales ventajas de utilizar un DBMS en este tipo de entornos se encuentran:

- **Control centralizado de la información:** Todas las operaciones y datos están centralizados bajo un sistema único, lo que mejora la seguridad, integridad y consistencia de la información.
- **Compacto:** Almacena los datos digitalmente, eliminando la necesidad de documentos físicos que ocupan espacio y pueden extraviarse.
- **Rápido:** El procesamiento de datos mediante una máquina es significativamente más veloz que cualquier proceso manual.
- **Menos laborioso:** Elimina tareas repetitivas y tediosas como la clasificación, búsqueda o actualización manual de archivos.
- **Información siempre actualizada:** Se dispone en todo momento de información precisa y en tiempo real.

En los entornos multiusuario, estas ventajas se potencian, ya que múltiples usuarios pueden acceder y modificar la base de datos simultáneamente sin comprometer la integridad de la información. Este tipo de arquitectura requiere de mecanismos que aseguren que cada operación sea registrada, controlada y recuperable en caso de error o fallo, lo que nos lleva a la importancia de las bitácoras.





Actividad Interactiva #1

Mira el video del link presentado a continuación y escribe en las líneas de abajo un resumen de 4 líneas de lo que consideres más importante.

<https://www.youtube.com/watch?v=Z1K5TV-bXCM>

Sección 2: Funciones específicas de las bitácoras

Las bitácoras (también llamadas logs de transacción) son estructuras de almacenamiento ampliamente utilizadas para registrar todas las modificaciones realizadas a una base de datos. Cada registro en la bitácora representa una operación específica de escritura y contiene información esencial:

- **Nombre de la transacción:** Identifica el proceso o conjunto de instrucciones responsables del cambio.
- **Valor antiguo:** Estado del dato antes de la modificación.
- **Valor nuevo:** Estado del dato después de la modificación.

Es fundamental que cada modificación de datos sea registrada en la bitácora antes de realizarse físicamente en la base de datos. Esta práctica asegura la posibilidad de recuperación ante errores o fallos del sistema.

Deshacer modificaciones (Rollback)

Gracias al almacenamiento del valor antiguo, es posible revertir una transacción en caso de ser necesario, devolviendo los datos a su estado anterior. Esta capacidad es crucial para garantizar la integridad de la base de datos cuando ocurre un fallo inesperado o cuando una transacción es cancelada.

Requisitos técnicos

Los registros de bitácora deben almacenarse en una memoria estable o permanente (como disco duro o SSD), lo que significa que el volumen de información puede crecer considerablemente con el tiempo. Por tanto, una buena gestión del almacenamiento de bitácoras es vital.

Puntos de sincronización y transacciones

Los comandos COMMIT y ROLLBACK definen lo que se conoce como puntos de sincronización. Estos marcan los límites entre dos transacciones consecutivas o el final de una operación lógica, y garantizan que la base de datos permanezca en un estado consistente. Existen tres momentos que pueden establecer un punto de sincronización:

- **COMMIT:** Confirma y guarda permanentemente todas las modificaciones realizadas desde el último punto de sincronización.
- **ROLLBACK:** Revierte todas las modificaciones realizadas desde el último punto de sincronización.
- **Inicio de un nuevo programa:** Define el comienzo de una nueva unidad lógica de trabajo.



Cuando se alcanza un punto de sincronización:

- Se confirman o revierten todas las modificaciones realizadas.
- Se pierden los posicionamientos actuales en la base de datos.
- Se liberan todos los registros bloqueados.

Es importante destacar que COMMIT y ROLLBACK finalizan una transacción, pero no necesariamente terminan el programa que las ejecutó.

Sección 3: Recuperación (rollback)

La recuperación (rollback) es un mecanismo esencial en los sistemas de gestión de bases de datos como PostgreSQL, que permite mantener la integridad, consistencia y confiabilidad de los datos, incluso ante errores inesperados. Este proceso es parte fundamental del modelo de manejo de transacciones del DBMS, y garantiza que una operación incompleta o fallida no altere el estado consistente de la base de datos.

Un rollback revierte todas las modificaciones realizadas por una transacción desde su inicio hasta el momento del fallo o error. Esta capacidad es vital para mantener la integridad y consistencia de los datos, especialmente cuando ocurren fallos como:

- Errores humanos.
- Fallos de aplicación.
- Fallos de hardware o del servidor.
- Interrupciones inesperadas del servicio.

Rollback en SQL

En lenguajes como SQL, se utiliza el comando:

ROLLBACK;

Este comando descarta todos los cambios desde la última sentencia BEGIN WORK o START TRANSACTION, devolviendo la base de datos a su estado anterior. Además, libera cualquier savepoint (puntos intermedios de control) que se haya establecido.

Rollback y concurrencia

En la mayoría de los sistemas de gestión de bases de datos relacionales (RDBMS), los rollbacks son específicos de la conexión, lo que garantiza que un rollback en una sesión no afecta a otras sesiones activas. Esta independencia permite una mejor gestión de la concurrencia y del aislamiento entre transacciones.

Funcionamiento de Rollback en PostgreSQL

PostgreSQL maneja las transacciones bajo el principio ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). Dentro de este modelo, el ROLLBACK asegura la atomicidad, es decir, que todas las operaciones de una transacción deben completarse correctamente, o ninguna debe aplicarse.



El funcionamiento básico es:

- Se inicia una transacción con BEGIN; o START TRANSACTION;
- Se ejecutan una o más sentencias SQL.
- Si alguna sentencia falla o el usuario lo decide, se ejecuta ROLLBACK;, cancelando todas las operaciones realizadas en esa transacción.

BEGIN;

UPDATE cuentas SET balance = balance - 200 WHERE nombre = 'Pepe';

UPDATE cuentas SET balance = balance + 200 WHERE nombre = 'Juan';

-- Error inesperado aquí

ROLLBACK;

Resultado: Ningún cambio se aplica a la base de datos.

Uso de SAVEPOINT para control parcial

PostgreSQL también permite definir SAVEPOINTS dentro de una transacción para realizar rollbacks parciales, es decir, revertir solo hasta un punto intermedio sin cancelar toda la transacción.

BEGIN;

INSERT INTO clientes(nombre) VALUES ('Pedro');

SAVEPOINT insercion_cliente;

INSERT INTO pedidos(cliente_id, producto_id) VALUES (999, 4); -- Error: cliente 999 no existe

ROLLBACK TO SAVEPOINT insercion_cliente;

COMMIT;

En este ejemplo:

- Se cancela la inserción fallida en pedidos.
- Pero se conserva la inserción del nuevo cliente.

Importancia de los logs y la recuperación automática

PostgreSQL registra todos los cambios en un archivo llamado Write Ahead Log (WAL). Este registro se utiliza para:

- Realizar rollback automático en caso de fallo.
- Recuperar el estado anterior en caso de corrupción o apagado forzado.
- Replicar cambios en sistemas en alta disponibilidad.

Cuando el servidor se reinicia después de un fallo, PostgreSQL lee los WAL para aplicar o deshacer transacciones pendientes, garantizando que la base de datos quede en un estado consistente.



Transacciones y Permanencia (COMMIT)

Una transacción agrupa varias operaciones SQL en una unidad lógica, todas deben ejecutarse correctamente y si alguna falla, se deshacen todas.

Una vez que la transacción se completa satisfactoriamente, se ejecuta el comando:

```
COMMIT;
```

Esto indica que los cambios se confirman de forma permanente en la base de datos y desde ese momento no se pueden deshacer con ROLLBACK.

Autocommit en PostgreSQL

Por defecto, PostgreSQL trabaja en modo AUTOCOMMIT activado, es decir, cada sentencia SQL ejecutada se considera una transacción independiente y se confirma automáticamente.

Para trabajar con transacciones manuales (BEGIN, COMMIT, ROLLBACK), debes desactivar el modo autocommit o usar bloques explícitos:

```
BEGIN;
```

```
-- operaciones
```

```
COMMIT;
```

Ejemplo completo de transacción con COMMIT

```
BEGIN;
```

```
-- Insertamos una nueva cuenta
```

```
INSERT INTO cuentas (n_cuenta, nombre, balance)  
VALUES (0679259, 'Pepe', 200);
```

```
-- Transferimos dinero de Pepe a Juan
```

```
UPDATE cuentas SET balance = balance - 137.00 WHERE nombre = 'Pepe';
```

```
UPDATE cuentas SET balance = balance + 137.00 WHERE nombre = 'Juan';
```

```
-- Verificamos balances
```

```
SELECT nombre, balance FROM cuentas WHERE nombre IN ('Pepe', 'Juan');
```

```
-- Confirmamos todos los cambios
```

```
COMMIT;
```



Ejemplo con ROLLBACK

BEGIN;

UPDATE cuentas SET balance = balance - 500 WHERE nombre = 'Ana';

UPDATE cuentas SET balance = balance + 500 WHERE nombre = 'Luis';

-- Suponemos que ocurre un error (por ejemplo, cuenta de Luis no existe)

-- Entonces revertimos la transacción

ROLLBACK;

Ninguno de los balances se verá afectado.

Actividad Interactiva #2

Ejercicio de asociación.

A continuación, verás una lista de términos y una lista de definiciones. Asocia Cada Término con su Definición:
Une cada término con la definición correcta.

Términos:

1. Valor antiguo
2. Valor nuevo
3. ROLLBACK
4. COMMIT



Definiciones:

- A. Acción que confirma y guarda permanentemente los cambios en la base de datos.
- B. Estado del dato antes de que se realice una modificación.
- C. Estado del dato después de que se ha aplicado una modificación.
- D. Operación que revierte los cambios realizados, devolviendo los datos a su estado previo.

Instrucciones: Asocia cada término (1-4) con la definición correspondiente (A - D). Y escríbela a continuación.
Ej: "1 - B"

Sección 4: Comandos de activación de los modos de operación

Los modos de operación de un Sistema Manejador de Bases de Datos (DBMS) permiten gestionar de forma eficiente la información, facilitando tareas como el ingreso, modificación, eliminación y recuperación de datos. Comprender cómo funcionan estos modos es esencial para mantener la integridad y disponibilidad de los datos, así como para responder adecuadamente ante fallos o pérdidas de información.

Funciones principales de un DBMS

Un DBMS no solo almacena datos, también gestiona cómo se organizan, manipulan y protegen.



Las funciones clave incluyen:

- Descripción de la base de datos:
- Elementos de datos
- Estructura de datos
- Interrelaciones y validaciones (Lenguaje de Definición de Datos - DDL)

Uso de datos:

- Acceso a la BD con permisos de administrador
- Manipulación de datos (DML):
 - Creación
 - Consulta
 - Actualización
 - Eliminación
- Respaldo de seguridad y recuperación de datos (Backup & Recovery)

Modo Alta (Inserción de Registros)

Este modo se refiere a la acción de añadir nuevos registros a una base de datos.

- Similar a añadir datos a un archivo secuencial, los datos se almacenan uno tras otro, siguiendo un orden específico. Cuando se necesita agregar un nuevo registro, este se añade únicamente al final del archivo.
- Los nuevos registros se agregan al final, no permite inserciones intermedias sin rehacer el archivo completo, ya que los registros se almacenan en un orden lineal fijo.
- Si se desea insertar registros entre otros ya existentes, se debe crear una copia actualizada del archivo original, que incluya todos los registros en el nuevo orden.

Modo Baja

Existen dos métodos para dar de baja a un registro en un archivo secuencial, donde no es fácil eliminar un registro situado en el interior de una secuencia, para ello se pueden seguir dos métodos:

- **Archivo auxiliar:** Utilizar y por tanto crear un segundo archivo auxiliar transitorio, también secuencial, copia del que se trata de actualizar. Se lee el archivo completo registro a registro y en función de su lectura se decide si el registro se debe dar de baja o no. En caso afirmativo, se omite la escritura en el archivo auxiliar. Si el registro no se va a dar de baja, este registro se reescribe en el archivo auxiliar. Tras terminar la lectura del archivo original, se tendrán dos archivos: original (o maestro) y auxiliar. El proceso de bajas del archivo concluye borrando el archivo original y cambiando el nombre del archivo auxiliar por el del inicial.
- **Marca lógica (flag):** Guardar o señalar los registros que se desean dar de baja con un indicador o bandera que se guarda en un array; de esta forma los registros no son borrados físicamente, sino que son considerados como inexistentes. Inevitablemente, cada cierto tiempo, habrá que crear un nuevo archivo secuencial con el mismo nombre, en el que los registros marcados no se grabarán.



Modo Recovery

El modo recovery o de recuperación es una de las funciones más importantes dentro de un DBMS, ya que permite restaurar la base de datos a un estado funcional tras una pérdida de datos, un error humano, una falla del sistema o cualquier tipo de incidente que afecte la integridad de la información. La tarea principal del administrador de base de datos en este contexto es diseñar, implementar y mantener una estrategia adecuada de respaldo y recuperación. Esta estrategia debe garantizar que, ante cualquier eventualidad, los datos puedan ser recuperados de forma segura, completa y en el menor tiempo posible. Para lograrlo, es fundamental contar con mecanismos de respaldo (backups) configurados de manera adecuada y con políticas de recuperación bien definidas.

Entre las tareas de administración de la seguridad de los datos, se encuentra la protección permanente de la información, la planificación y prueba de respuesta ante distintos tipos de fallos, así como el monitoreo constante de los procesos de copia de seguridad y restauración. Los posibles escenarios que pueden generar una pérdida de datos incluyen fallas en medios físicos como discos duros o memorias, errores cometidos por los usuarios, fallos en las aplicaciones que manipulan los datos o incluso desastres naturales. Por esta razón, los DBMS deben estar preparados para responder rápidamente ante cualquiera de estos eventos. Una estrategia sólida de recuperación debe incluir copias de seguridad completas e incrementales, así como el uso de registros de transacciones que permitan reconstruir las operaciones realizadas antes de la falla.

Comandos de Activación de los Modos de Operación

Para hacer uso de los diferentes comandos correspondientes a un modo de operación en un Sistema Manejador de Bases de Datos (DBMS), es necesario estar conectado como administrador o haber asumido un rol que incluya el perfil de derechos "Service Management". Esto se debe a que estos comandos afectan directamente el estado interno de la base de datos, su disponibilidad, y la manera en que se accede a ella, por lo tanto, requieren privilegios especiales para garantizar la seguridad y el control del sistema.

Comando STARTUP

El comando STARTUP se utiliza para iniciar una base de datos, y consta de tres fases principales. Cada una de estas fases cumple una función específica dentro del proceso de arranque del sistema, y se puede activar agregando una opción específica al comando STARTUP, estas fases son:

- **Fase de No Montaje (NOMOUNT):** En esta fase, se leen los parámetros del sistema, se inicializan las estructuras de memoria (como la memoria compartida SGA y la memoria PGA para procesos individuales), y se activan los procesos de segundo plano, los cuales se encargan de tareas internas como la escritura de datos en disco, la recuperación y otras operaciones automáticas del DBMS.

Sin embargo, en esta fase la instancia se arranca sin estar asociada aún a la base de datos, lo que significa que no se han cargado los archivos de control ni los datos. Esta fase se utiliza generalmente para tareas de mantenimiento, como la creación o modificación del archivo de control, o cuando es necesario reconstruir la base de datos. El comando utilizado es:

STARTUP NOMOUNT;



- **Fase de Montaje (MOUNT):** Durante la fase de montaje, la instancia ya se asocia a la base de datos mediante el uso de los archivos de parámetros que permiten localizar los archivos de control. Estos archivos contienen información crítica, como el nombre de los archivos de datos y los archivos de registro de rehacer (redo logs), que registran todos los cambios realizados en la base de datos para poder recuperarlos en caso de falla.

Es importante destacar que, aunque la instancia ya está conectada a la base de datos, los archivos de datos y los registros de rehacer aún no están abiertos, por lo que no están disponibles para los usuarios finales. Esta fase se utiliza comúnmente para realizar tareas administrativas, como la recuperación de datos o la ejecución de ciertas configuraciones antes de abrir completamente la base de datos. Para esta fase se pueden usar los comandos:

STARTUP MOUNT;

ALTER DATABASE MOUNT;

- **Fase de Apertura (OPEN):** En la fase de apertura, se abren los archivos de datos y los archivos de registros de rehacer, permitiendo que la base de datos esté completamente funcional y disponible para las operaciones normales de los usuarios, como consultas, inserciones, actualizaciones y eliminaciones. Es decir, la base de datos ya está completamente operativa en este estado.

Es importante mencionar que si no existen registros de rehacer válidos (por ejemplo, si han sido dañados o eliminados), es necesario utilizar la opción resetlogs, que indica al sistema que debe crear un nuevo conjunto de registros de rehacer, lo que reinicia el historial de transacciones. Esta opción debe usarse con precaución, ya que puede afectar la posibilidad de recuperar transacciones anteriores.

Los comandos disponibles para abrir la base de datos son:

STARTUP OPEN;

ALTER DATABASE OPEN;

- **Y si es necesario utilizar la opción de reinicio de registros de rehacer:**

STARTUP OPEN RESETLOGS;

ALTER DATABASE OPEN RESETLOGS;

- **Además, existen otras variantes del comando STARTUP** que permiten controlar quién puede conectarse o forzar un arranque en situaciones especiales. Por ejemplo, el comando: STARTUP RESTRICT; permite que solo los usuarios con el privilegio RESTRICTED SESSION puedan conectarse. Esto es útil cuando se quiere hacer mantenimiento sin interferencia de usuarios. Por otro lado: STARTUP FORCE; se utiliza para forzar el arranque de la base de datos, realizando internamente un SHUTDOWN ABORT (cierre abrupto) y luego arrancando la base de datos desde cero. Es útil cuando el sistema no responde y no se puede hacer un cierre limpio.

Comando SHUTDOWN

El comando SHUTDOWN se utiliza para detener o apagar una base de datos de forma controlada o inmediata, según el tipo de cierre que se necesite. Esta operación también requiere privilegios de administrador, ya que detiene el acceso de los usuarios, desconecta sesiones activas y puede afectar las transacciones en curso. Existen varias cláusulas que determinan cómo se ejecuta este cierre:



- **Shutdown Normal:** Este es el método por defecto. Durante este tipo de parada, el sistema no permite nuevas conexiones, pero espera a que las sesiones actuales finalicen voluntariamente su trabajo y se desconecten. No se fuerza la desconexión ni se pierde información, por lo que no se requiere recuperación al reiniciar la base de datos. Este método es el más seguro y ordenado, pero puede tardar más si los usuarios no cierran sus sesiones rápidamente.
SHUTDOWN;
- **Shutdown Immediate:** En esta modalidad, el sistema no permite nuevas conexiones y desconecta inmediatamente las sesiones activas. Todas las transacciones que no hayan sido confirmadas (no se les haya hecho COMMIT) serán revertidas automáticamente mediante un ROLLBACK. Este tipo de cierre es más rápido que el normal y también garantiza que no se pierda integridad en la base de datos. En el próximo arranque no se requerirá recuperación adicional.
SHUTDOWN IMMEDIATE;
- **Shutdown Transactional:** Esta opción permite que la base de datos se detenga únicamente cuando todas las transacciones activas hayan terminado. No permite nuevas conexiones ni nuevas transacciones, pero espera que las que ya están en curso finalicen, después de lo cual desconecta a los usuarios de forma ordenada. Este método es útil cuando se necesita detener la base de datos sin interrumpir procesos críticos en curso. Al igual que los métodos anteriores, no requiere recuperación al volver a iniciar.
SHUTDOWN TRANSACTIONAL;
- **Shutdown Abort:** Este tipo de cierre aborta inmediatamente todos los procesos de la base de datos, desconectando sin esperar ni permitir la finalización de sesiones ni de transacciones. Las transacciones que no estén confirmadas se perderán y deberán ser recuperadas al próximo arranque, ya que el sistema se verá obligado a aplicar procedimientos de recuperación automática. Aunque es la forma más rápida de apagar la base de datos, también es la más riesgosa, ya que puede provocar corrupción de datos si se utiliza de forma inadecuada. Solo debe usarse en casos de emergencia, como cuando el sistema no responde a otros tipos de cierre.
SHUTDOWN ABORT;

